

The Unreasonable Effectiveness of Philosophy in Computer Science

Fabio Zanasi



About me

BA in
Philosophy



MSc in
Logic



PhD in
Computer Science



About me

BA in
Philosophy



Stanisław Leśniewski
(1886 - 1939)



Leading representative of the **Lwów–Warsaw School of Logic**

Author of an alternative foundation of mathematics
based on a **nominalist** logical formalism.

His only PhD student was **Alfred Tarski**, whose semantics
drew much inspiration from Leśniewski's work.

About me

BA in
Philosophy



Stanisław Leśniewski
(1886 - 1939)



My Bachelor thesis was a study of
creative definitions in Leśniewski's logic.

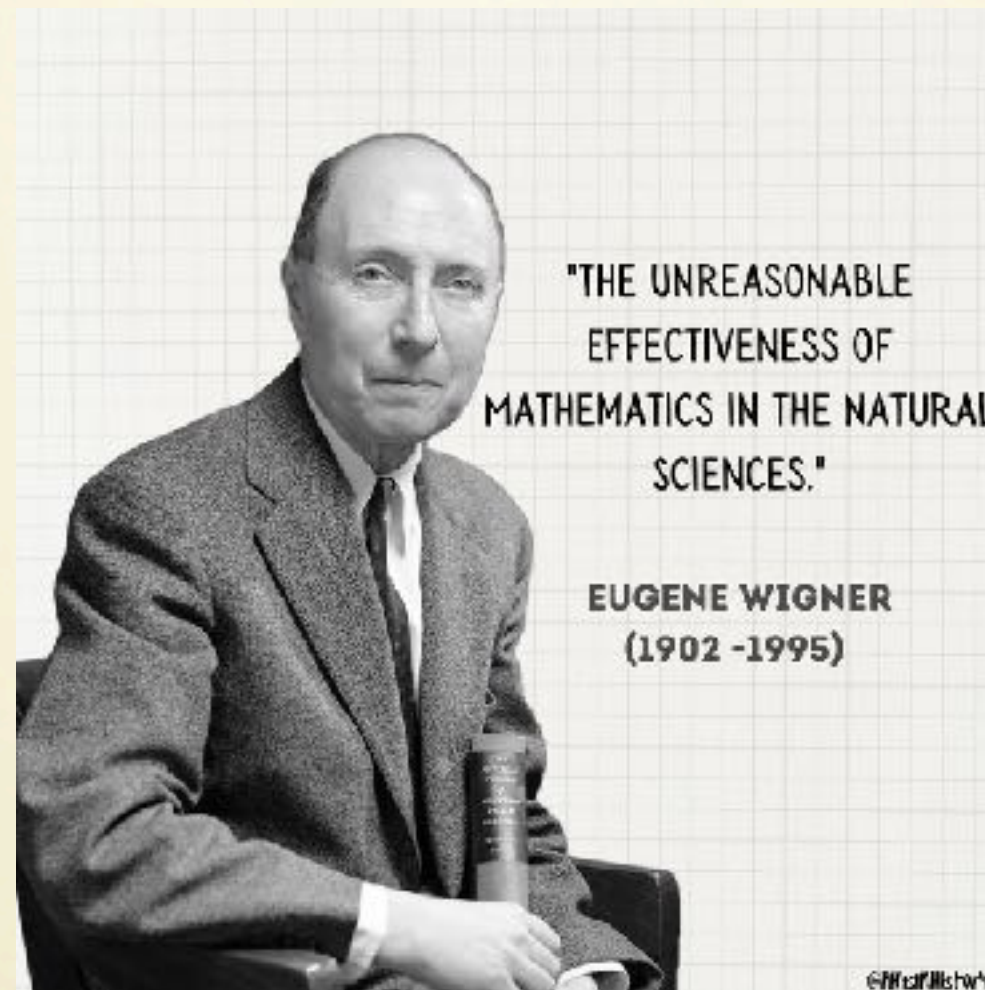
To know more (on my website):

[**On Creative Definitions in Leśniewski's Ontology**](#)

Fabio Zanasì

Joint work with F.V. Riskey

About today



Computer Science was born from Philosophy.

Still today, Philosophy can guide us understanding
(and doing `better') Computer Science

About today

Argue in three ways:

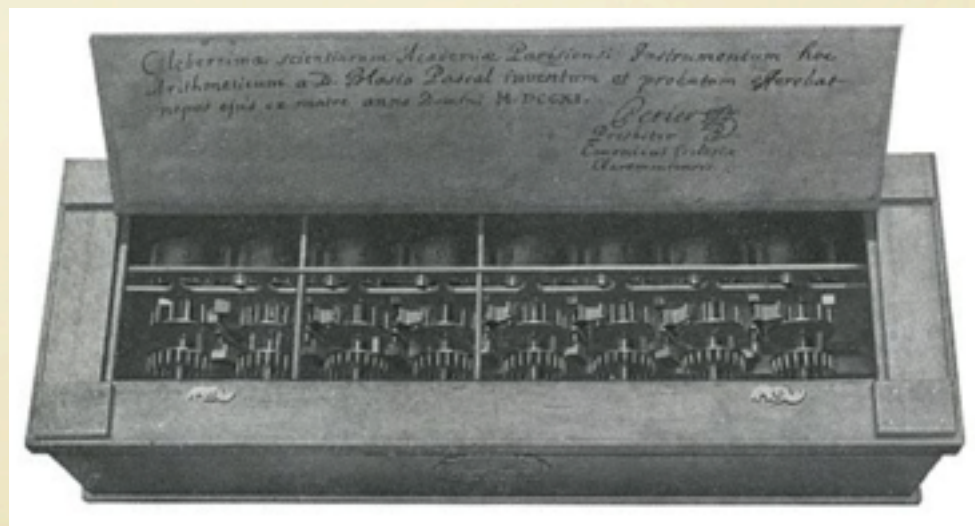
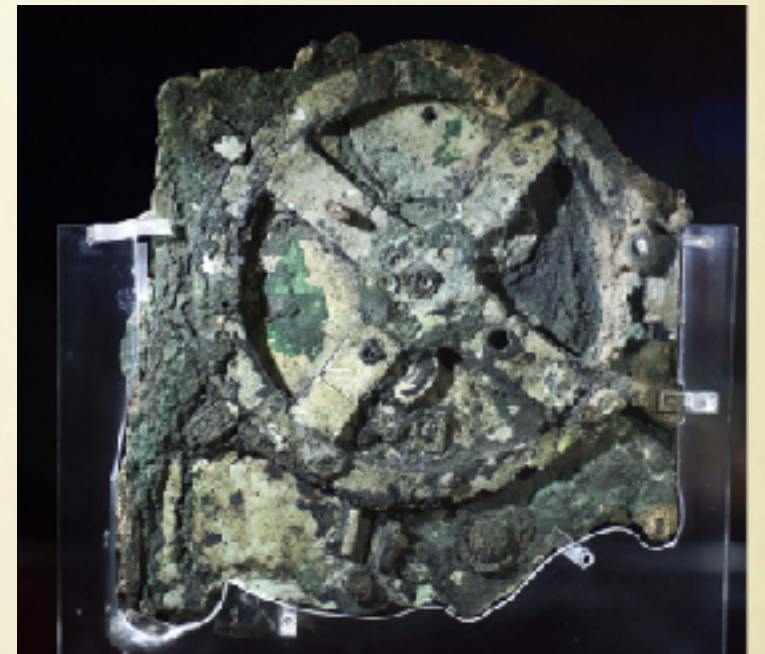
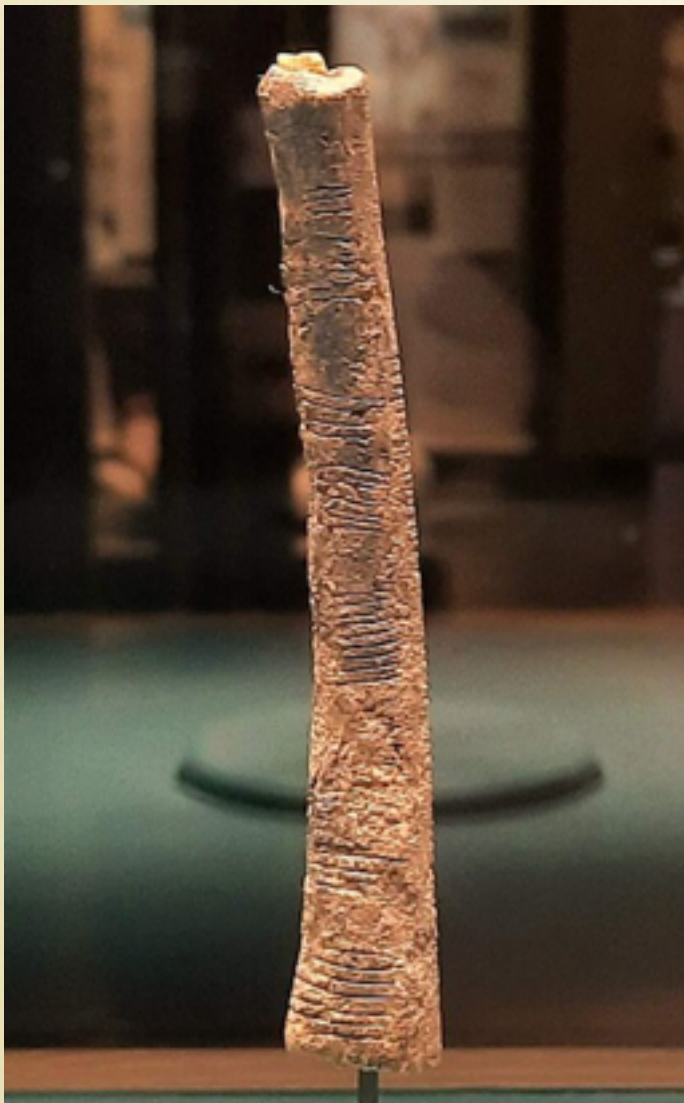
1. How Computer Science was born from Philosophy.
2. How fundamental constructs of programming languages (types, variables) can be understood through Philosophy of Language and Ontology.
3. How recent developments in Formal Modelling in Computer Science embody a century old debate on the nature of logical reasoning and its representation.

1.

From Philosophy to Computer Science

When was Computer Science born?

People have used computing devices for millennia, but none of them were **programmable** or **universal**



When was Computer Science born?

Alan Turing (1912-1954) is the founding father of Computer Science

The **Turing Machine** is the first computational model that is universal and programmable.

Turing uses it to prove that there are **undecidable** problems (*decades before* the first computer was built!)

The **Church-Turing thesis** asserts that no algorithm cannot be simulated by a Turing machine.



When was Computer Science born?

It all started here:

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions

What has this to do with Philosophy?

Turing at Cambridge

It all started here:



Cambridge in the 1930's:



B. Russell

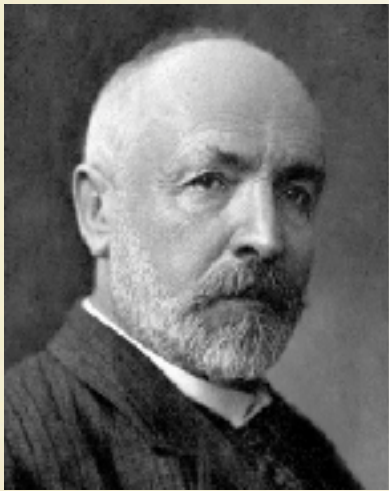


L. Wittgenstein

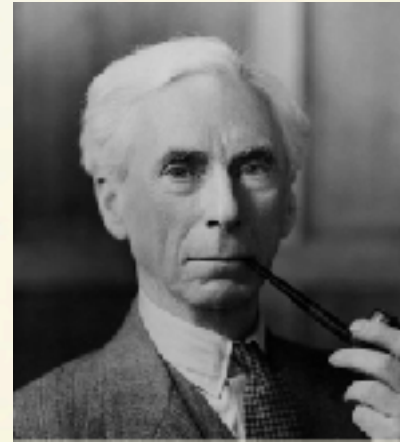


F. Ramsey

Foundational Crisis of Mathematics



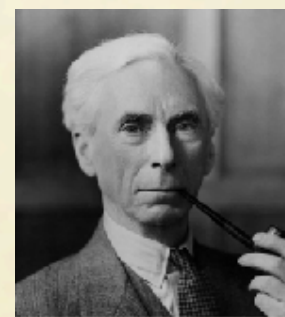
Cantor's Set Theory
(1880's)



Russel's Paradox
(1900's)



Gödel's Incompleteness
(1930's)



Logicism

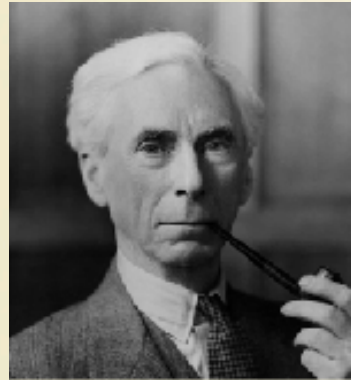


Formalism



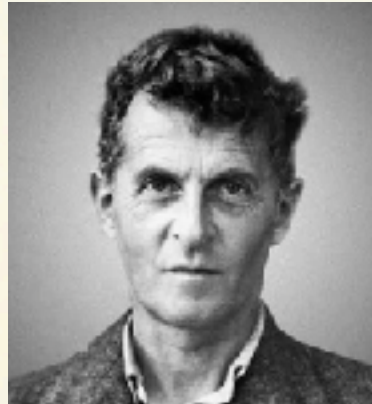
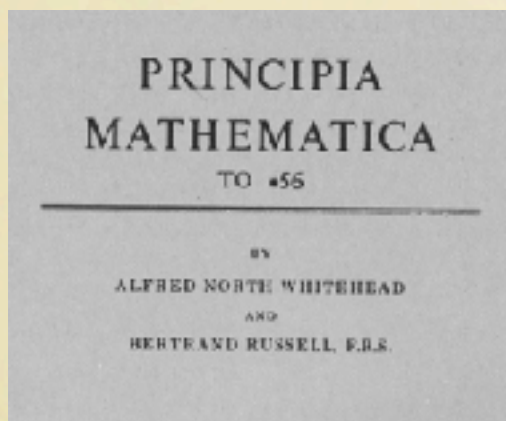
Intuitionism

Influences on Turing



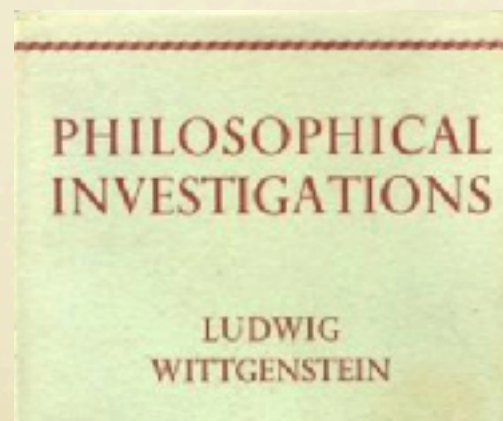
B. Russell

*reduce all
mathematics to
precise symbolic
manipulation*



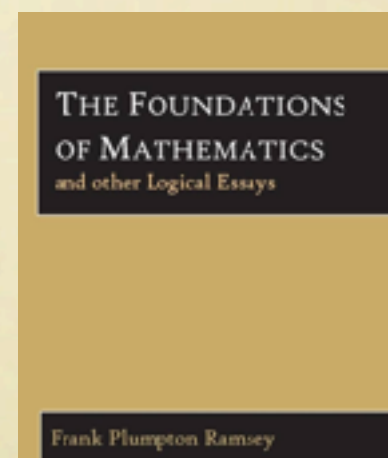
L. Wittgenstein

*mathematics is a human
practice governed by
rules, not a completed
logical system.*



F. Ramsey

*what is the method
by which a logical
conclusion is
reached?*



Turing: Logicism or Formalism?



Turing explicitly rejected key assumptions of logicism:

He did not think mathematical truth was reducible to self-evident logical truths.

He treated mathematical propositions as things we might or might not be able to decide, not eternal logical facts.

Turing: Logicism or Formalism?



The Hilbert formalist program saw mathematics as symbolic manipulation within formal systems.

Turing *did* embrace symbol manipulation. However, his focus is not on “What is the nature of mathematical objects?” but on “What is a calculation?” (How we compute with objects)

Turing’s procedural formalism:
mathematics as rule-following computation

Turing vs Wittgenstein



Nature of Mathematics

Human practice

Mechanical procedures

Role of Foundations

Clarify how we use
mathematical expressions

Define effective
methods rigorously

2. The Ontology of Programming

Types in Programming

A type tells you what **kind of data** you're working with.

C++

```
float temperature = 36.5;
```

Rust

```
let count: i32 = 10;
```

Java

```
int count = 5;  
double price = 19.99;  
String name = "Alice";  
  
public int add(int a, int b) {  
    return a + b;  
}
```


Types in Programming

A type tells you what **kind of data** you're working with.

C++

```
float temperature = 36.5;
```

Rust

```
let count: i32 = 10;
```

Java

```
int count = 5;  
double price = 19.99;  
String name = "Alice";  
  
public int add(int a, int b) {  
    return a + b;  
}
```

Why Types?

```
int count = 5;  
double price = 19.99;  
String name = "Alice";  
  
public int add(int a, int b) {  
    return a + b;  
}
```

Types encode programmer intentions

Types catch errors before the program runs

Types help compilers generate efficient code

Teaser: our typing discipline reveals the ontological commitment of our language

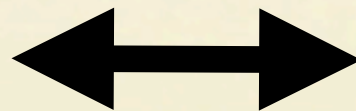
The origin of types



Turing
Machine



Alonzo
Church
(1903-1995)



λ -calculus

The λ -calculus à la Church

λ -terms:

Variables

$x : T, y : T', z : T'', \dots$

Abstractions

$\lambda x : T. M$

Applications

$M N$

(+ some constants/
primitive functions)

Types:

Basic

$Nat, Bool, \dots$

Function types

$T \rightarrow T'$



Alonzo
Church
(1903-1995)

λ -calculus

Example:

$\lambda x : Nat. x + 1 : Nat$

has type $Nat \rightarrow Nat$

Typing à la Church

$\lambda x : \text{Nat} . x + 1$

The type declaration $x : \text{Nat}$ is part of the term.

Pros

- Type checking/inference are straightforward.
- Type errors are prevented.
- Program is self-documenting.

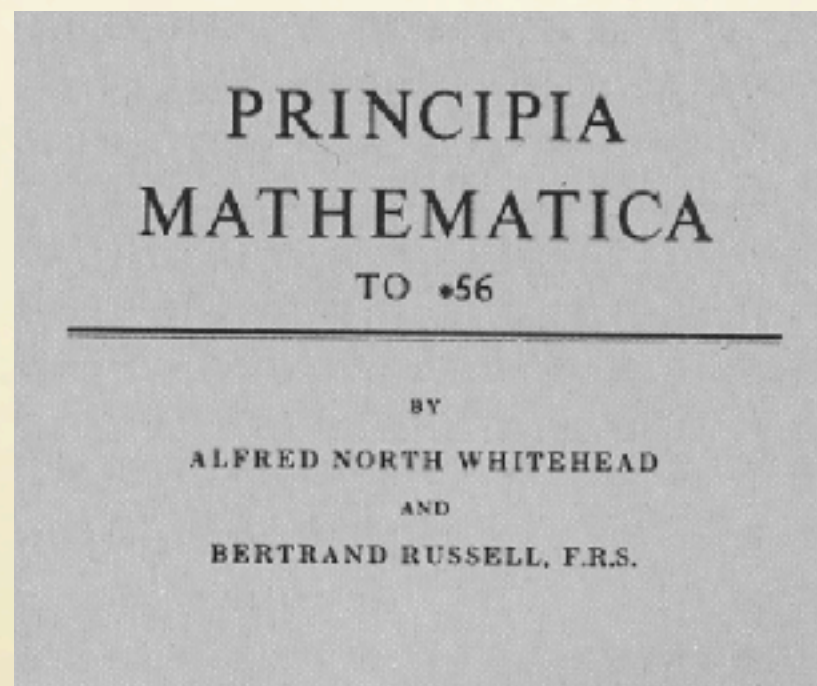
Cons

- Code is more verbose.
- Less flexibility: no polymorphism.

$\lambda x : \text{Nat} . x + 1$
 $\lambda x : \text{Float} . x + 1$
 $\lambda x : \text{Int} . x + 1$

The origin of types

Church's typing discipline is directly inspired to Russell's type theory, as exposed in *Principia Mathematica*.



The Philosophy of Typing

Church's typing discipline is directly inspired to Russell's type theory, as exposed in *Principia Mathematica*.

Types are a **logical safeguard** against paradoxes (program errors).

Types separate and stratify expressions into categories.

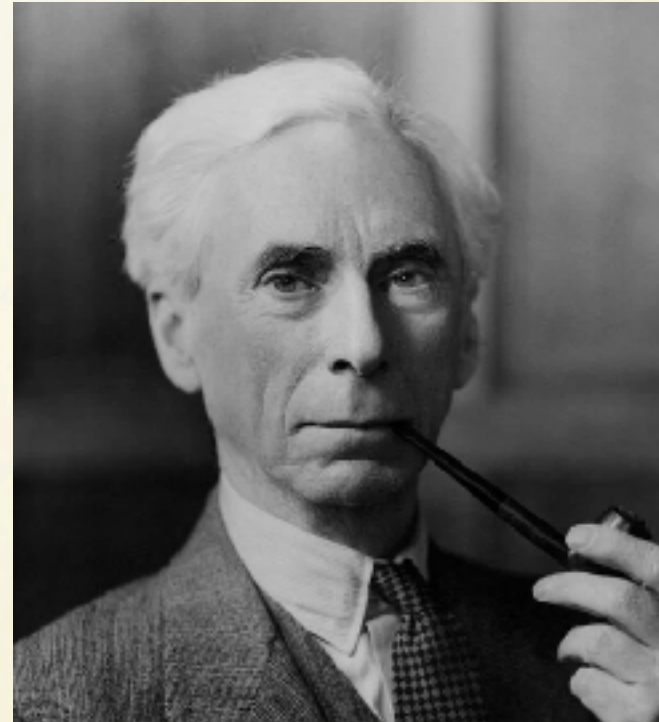
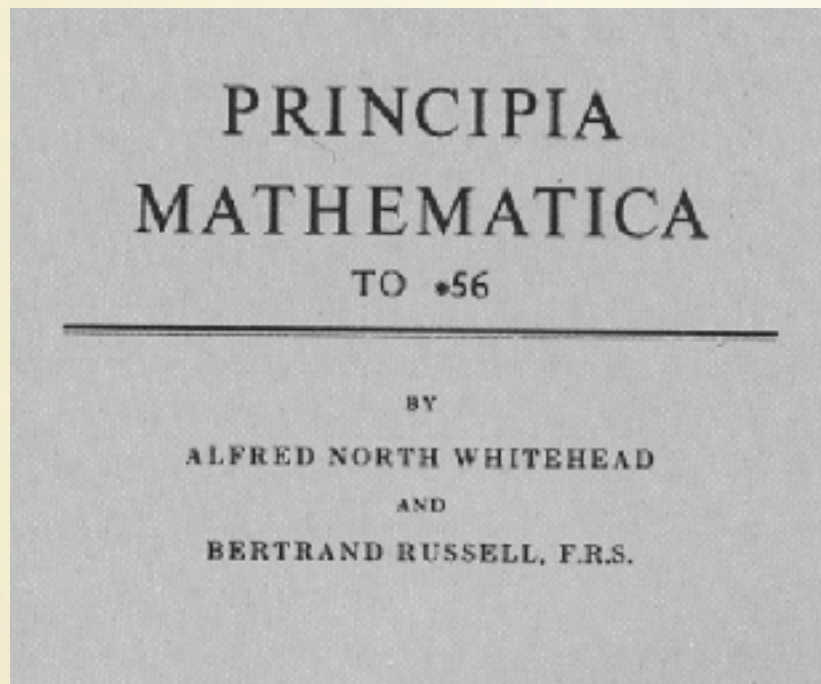
Ontological Commitment

In Church-Russell typing discipline, terms (expressions) are born with a type, and are **inseparable** from it.

Types tell us which entities *outside of the language* a term can stand for if the resulting expression is to be meaningful.

The Church-Russell paradigm of typing supports a **platonist view** of mathematical language.

Ontological Commitment



*Whatever may [...] occur in any true or false proposition [...] I call a **term**... I shall use as synonymous with it the words unit, individual, and **entity**. [...] a man, a moment, a number, a class, a relation, a chimaera, or anything else that can be mentioned, is sure to be a term.*

*[T]here is a **class of objects**, the **type** of x , all of which must also belong to the range of **significance** of $\phi(x)$, however ϕ may be varied.*

The λ -calculus à la Curry

λ -terms:

Variables

$x, y, z, \dots$

Abstractions

$\lambda x . M$

Applications

$M N$

(+ some constants/
primitive functions)



Haskell
Curry
(1900-1982)

Example:

$\lambda x . x + 1$

can be **polymorphic**:

$Nat \rightarrow Nat$

$Float \rightarrow Float$

$\dots$

Typing à la Curry

$$\lambda x . x + 1$$

The type declaration is not part of the term.

Pros

- Code is more concise and elegant.
- Polymorphism.
- Encourage 'exploratory' programming.

Cons

- Harder to debug.
- Non-trivial type checking/inference.
- Inefficient execution.

The Philosophy of Typing

Curry's typing discipline naturally fits a **nominalist view** of mathematical language.

Terms are just strings of symbols.

Types are human-imposed classifications.
They are *a posteriori* meta-information, not
part of the language ontology.

Nominalism in Mathematics

Mathematics is a **purely linguistic enterprise**, and its ontology must be kept strictly minimal.



Stanisław Leśniewski
(1886 - 1939)

Mathematical terms get meaning from use, not from reference to abstract objects.

Sets, classes, predicates, etc. do not exist as independent from concrete, individual entities.

No empty set. $\{x\} = x$.

Nominalism and Formalism



Curry adhered to **formalism**.

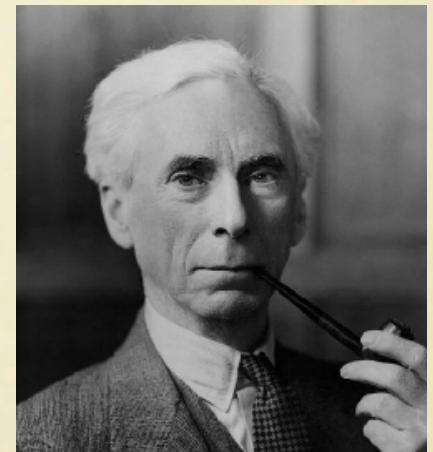
Formalism is the most natural way of implementing nominalism in the construction of a formal system.

Variables

Just as types, our treatment of **variables** in formal languages (including programming) reveal our ontological commitments.

Variables in Principia Mathematica:

We will adopt the following conventions. Variables of the lowest type occurring in any context will be denoted by small Latin letters [...]; a predicative function of an argument x [...] will be denoted by $\phi!x$ (where ψ , χ , θ , f , g , F , or G may replace ϕ)



Variables (surreptitiously) come with a type, determined by their font.

Variables as containers

Indications of a **platonist** view on variables in programming languages:

Variables are always introduced with an explicit type

```
var x : int  
var y : int
```

Variables behave as **containers** relative to assignment

```
var x = 4;   var y = x  
var x = 5;   print y
```

4

Variables as containers

Indications of a **platonist** view on variables in programming languages:

In the containers paradigm, there is a **1-to-1 correspondence** between variables and entities they denote.

var x $\longrightarrow$ 4

var y $\longrightarrow$ 5

Variables as pointers

Indications of a **nominalist** view on variables in programming languages:

Variables are **untyped**. Their type is inferred at execution.

Variables behave as **pointers** relative to assignment

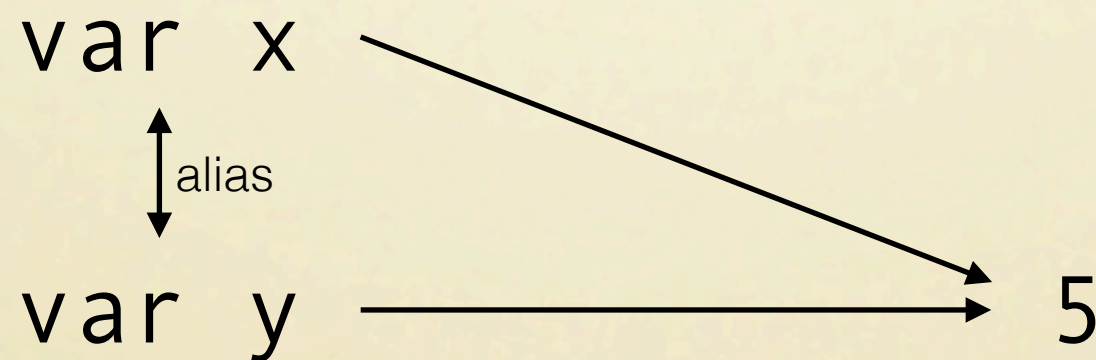
```
var x = 4;   var y = x  
var x = 5;   print y
```

5

Variables as pointers

Indications of a **nominalist** view on variables in programming languages:

in the pointers paradigm, we trade a minimal ontological commitment for more complexity in our language.



Variables for Nominalism

This is reminiscent of
Leśniewski's logical formalisms.

The type and meaning of language
terms (including variables) are not
predetermined, but recovered via a
context-sensitive, algorithmic analysis.

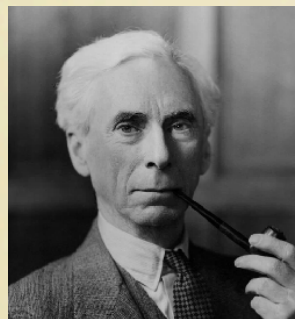


Summing up

Variables as
containers

Static typing

Platonism



Modern programming
languages often **mix** the
two paradigms

Variables as
pointers

Dynamic typing

Nominalism
(formalism)



3. The Diagrams Renaissance

Notation matters

*54.43. $\vdash \lambda \alpha, \beta \in 1. \supset \alpha \wedge \beta = \Lambda. \equiv \alpha \vee \beta \in 2$

Dem.

$\vdash$ *54.26. $\supset \vdash \lambda \alpha = t'x. \beta = t'y. \supset \alpha \vee \beta \in 2. \equiv x \neq y.$

[*51.231] $\equiv t'x \wedge t'y = \Lambda.$

[*13.12] $\equiv \alpha \wedge \beta = \Lambda$ (1)

$\vdash (1). *11.1135. \supset$

$\vdash \lambda (x, y). \alpha = t'x. \beta = t'y. \supset \alpha \vee \beta \in 2. \equiv \alpha \wedge \beta = \Lambda$ (2)

$\vdash (2). *11.54. *52.1. \supset \vdash$ Prop

From this proposition it will follow, when arithmetical addition has been defined, that $1 + 1 = 2$.

1	$\forall x \exists y [Px \wedge Qy]$	ISS.
2	$\exists y (Px \wedge Qy)$	1, ($\forall E$)
3	$Px \wedge Qy$	ass.
4	Px	3, ($\wedge E$)
5	Px	3-4, ($\exists E$)
6	$\forall x Px$	5, ($\forall I$)
7	$Px \wedge Qx$	ISS.
8	Qx	7, ($\wedge E$)
9	$\exists x Qx$	8, ($\exists I$)
10	$\exists x Qx$	7-9, ($\exists E$)
11	$\forall x Px \wedge \exists x Qx$	6, 10, ($\wedge I$)
12	$\forall x \exists y (Px \wedge Qy) \rightarrow \forall x Px \wedge \exists x Qx$	1-11, ($\rightarrow I$)

$$\frac{\frac{\frac{B_1[A \wedge B \wedge C, \neg A, \neg B]}{B_1[A \wedge B \wedge C, \neg A \wedge \neg B]}(\wedge I)}{\neg(A \wedge B), \neg(A \wedge C), (A \wedge B) \wedge C}(\wedge E) \quad \frac{B_2}{\neg(A \wedge B), \neg(A \wedge C), (A \wedge B) \wedge C}(\wedge E)}{\neg(A \wedge B), \neg(A \wedge C), (A \wedge B) \wedge C}(\wedge E)$$

where B_1 is the following derivation:

$$\frac{\frac{B_1[A \wedge B, \neg A]}{B_1[A \wedge B, \neg A]}(\wedge I) \quad \frac{B_2[A \wedge B, \neg A]}{B_2[A \wedge B, \neg A]}(\wedge I)}{\neg(A \wedge B), \neg(A \wedge C), (A \wedge B) \wedge C}(\wedge E) \quad \frac{B_3[A \wedge B, \neg A]}{B_3[A \wedge B, \neg A]}(\wedge I) \quad \frac{B_4[A \wedge B, \neg A]}{B_4[A \wedge B, \neg A]}(\wedge I)}{\neg(A \wedge B), \neg(A \wedge C), (A \wedge B) \wedge C}(\wedge E)$$

Historically, formal languages (from logic to programming) have predominantly been **textual**.

```

[case 0.5 (strength 0 "fade") ifrom 0.0 to 1.0 [case 0 " easing-fade-exp]
[eval (show-panel "game-over-panel")]
[case 0.5 (strength 0 "fade") ifrom 1.0 to 0.0 [case 0 " easing-fade-exp]
[case 2.0 (strength 0 "distortion") ifrom 1.0 to 0.0 [case 0 " easing-fade-exp]]

[action-list define-action-list hurt
[case 0.2 (strength 0 "distortion") ifrom 0.0 to 0.5 [case 0 " easing-fade-exp]
[case 0.5 (strength 0 "distortion") ifrom 0.5 to 0.0 [case 0 " easing-fade-exp]]

[action-list define-action-list transition
[setf (direction 0 "fade") 0.0]
[case 0.5 (strength 0 "fade") ifrom 0.0 to 1.0]
[delay 0.1]
[eval (loop for func = (pop (on-complete 0 "fade"))
while func
do (funcall func))]
[setf (direction 0 "fade") 1.0]
[delay 0.1]
[case 0.5 (strength 0 "fade") ifrom 1.0 to 0.0]]

[action-list define-action-list start-game

```

```

1 # User Defined Function to add Two Numbers
2 """ User Defined Function to add Two Numbers """
3 #
4 def addtwonum(a,b):
5     sum = a+b
6     return sum
7 #
8 # Main to call addtwonum Function
9 num1 = eval(input("Enter first number: "))
10 num2 = eval(input("Enter second number: "))
11 sum = num1 + num2
12 print(f"Addition sum {num1} + {num2} is = {sum}")
13

```

```

1 # User Defined Function to add Two Numbers
2 """ User Defined Function to add Two Numbers """
3 #
4 def addtwonum(a,b):
5     sum = a+b
6     return sum
7 #
8 # Main to call addtwonum Function
9 num1 = eval(input("Enter first number: "))
10 num2 = eval(input("Enter second number: "))
11 sum = num1 + num2
12 print(f"Addition sum {num1} + {num2} is = {sum}")
13

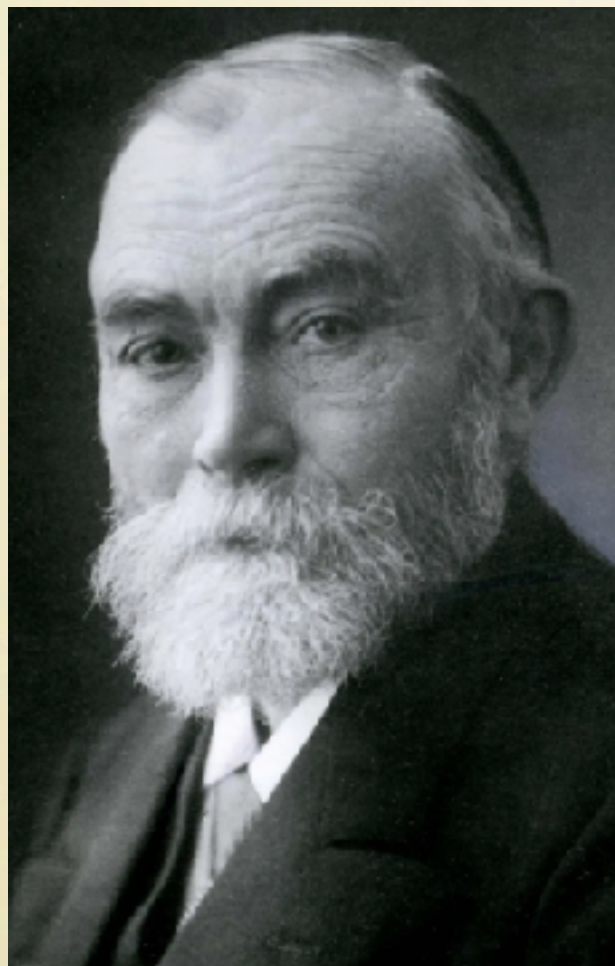
```

Why is logic (and programming) textual?

- Simplicity
- Abstractness
- Convenience
- Mechanisation

Frege's Begriffsschrift

Modern Logic is born with G. Frege (1848-1925)



Separate Logic
from Psychology

First rigorous
treatment of
quantifiers,
predicates,
logical
consequence.

Ideography

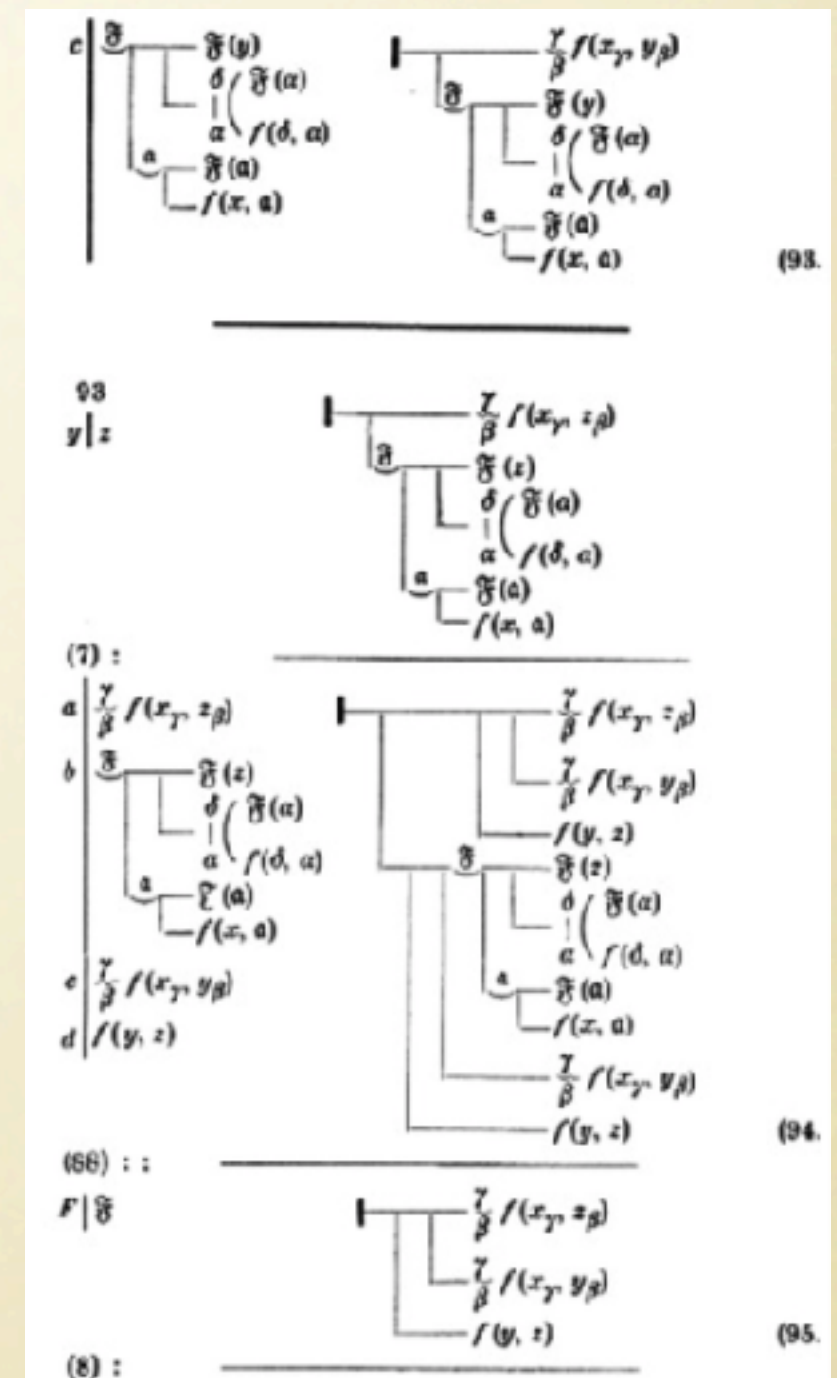
Interestingly, *Begriffsschrift* did contain a two-dimensional language, called *ideography*.

Vertical strokes mark the antecedent of implication.

Quantifiers are shown by 'hooks'.

Frege himself only thought of ideography as a prop for calculations, not as a standalone formalism.

Ideography didn't catch on, and is mostly forgotten.



Another precursor: C. S. Peirce

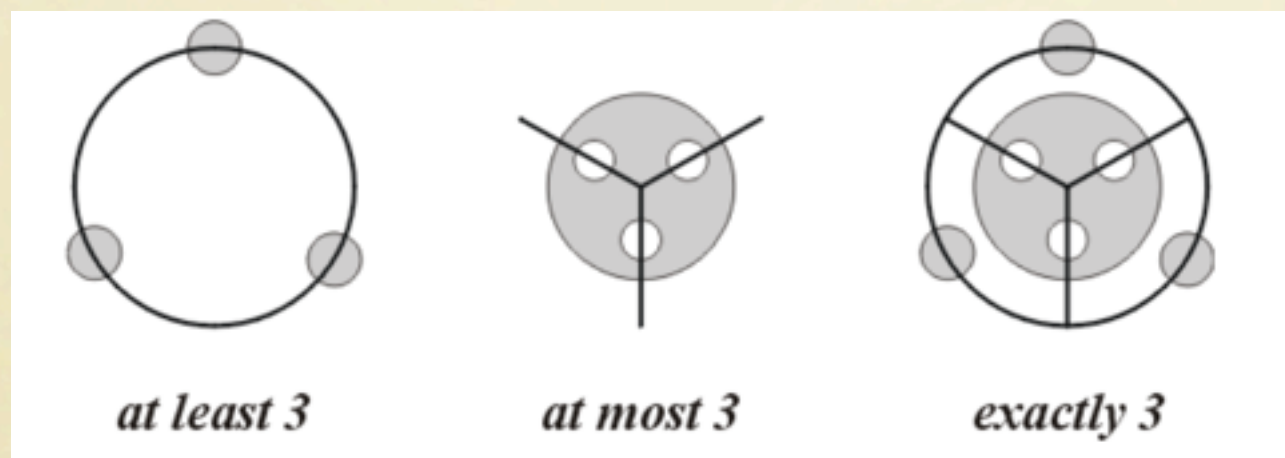
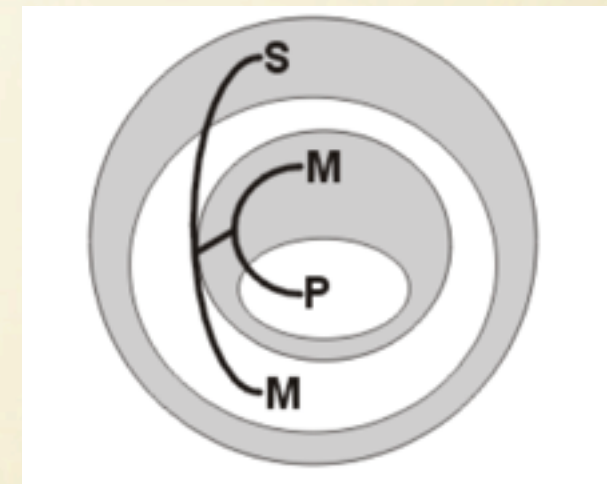
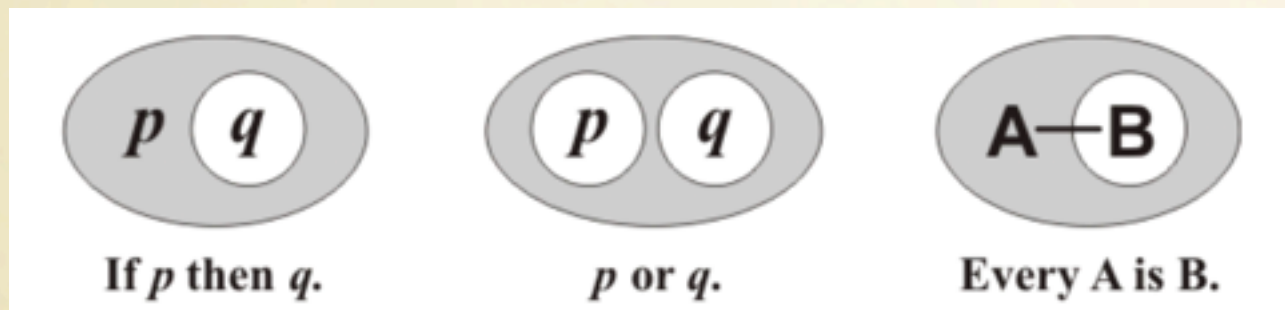
C. S. Peirce
(1839-1914) is a
contemporary of Frege.



- Peirce's work is grounded in **semiotic and psychology**.
- Goal of logic is to model **human inferential practices**.
- Best representation for logical thinking is manipulating signs in **space**, leveraging geometric, topological, combinatorial intuitions.
- Variable-free approach: **relations** between graphical entities (circles, lines, ...) are the fundamental units of the language.

Existential Graphs (1880s)

One of the most remarkable attempts to build a **visual logic**.



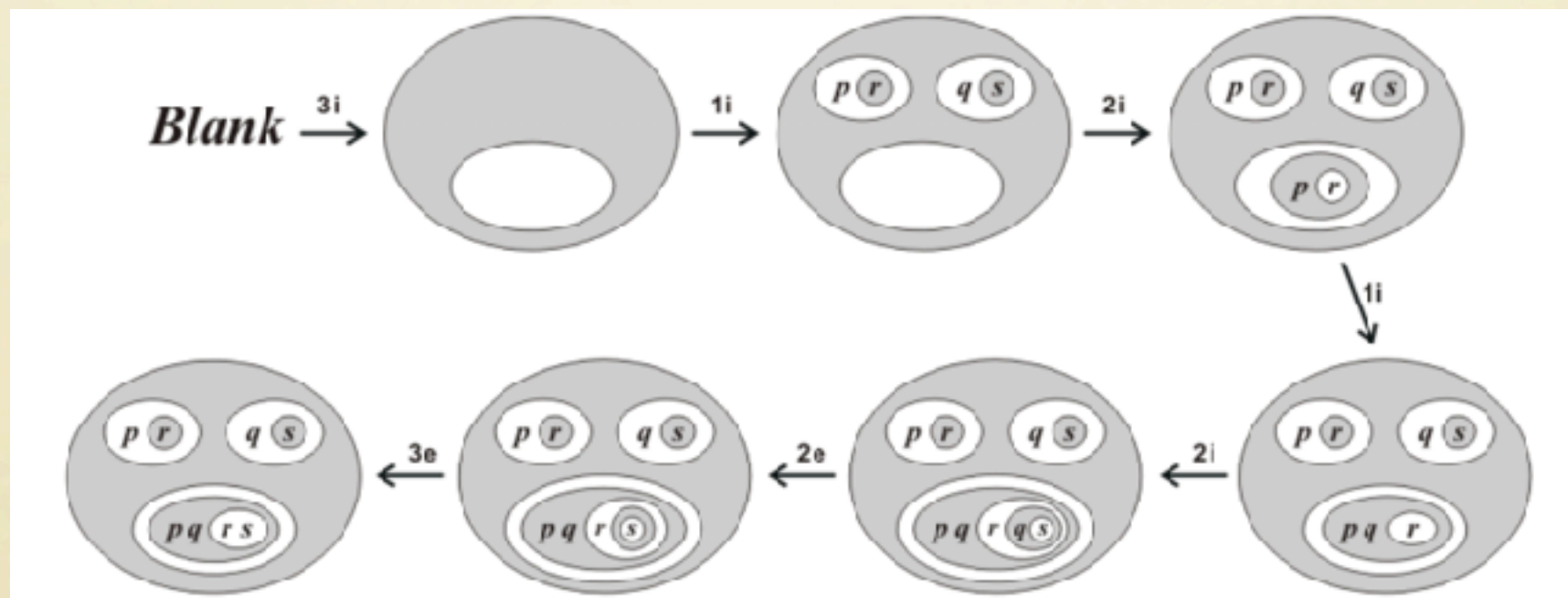
From J. Showa - Peirce's Tutorial on Existential Graphs

An Example

$$((p \supset r) \wedge (q \supset s)) \supset ((p \wedge q) \supset (r \wedge s))$$

In *Principia Mathematica*: 43-lines proof

With Existential Graphs:



From J. Showa - Peirce's Tutorial on Existential Graphs

Existential Graphs

When Peirce published about existential graphs, textual logical notation (especially thanks to Peano) had already become the standard.

$$\begin{aligned} &\exists 0 : 0 \in \mathbb{N} \\ &\forall n \in \mathbb{N} : \exists n' \in \mathbb{N} \\ &\neg(\exists n \in \mathbb{N} : n' = 0) \\ &\forall m, n \in \mathbb{N} : m' = n' \Rightarrow m = n \\ &\forall A \subseteq \mathbb{N} : (0 \in A \wedge (n \in A \Rightarrow n' \in A)) \Rightarrow A = \mathbb{N} \end{aligned}$$

Peirce's focus on semiotics and rather convoluted writing style didn't help making his formalism popular.

From Logic to Computer Science

Visual formal languages have largely disappeared for decades. Textual formalisms have become the standard in Computer Science too.

```
(case 0.5 (strength (u 'fade)) :from 0.0 :to 1.0 :ease #'easing-f:in-exp)
(eval (show-panel 'game-over-panel))
(case 0.5 (strength (u 'fade)) :from 1.0 :to 0.0 :ease #'easing-f:out-exp)
(case 2.0 (strength (u 'distortion)) :from 1.0 :to 0.0 :ease #'easing-f:in-circ))

(action-list:define-action-list hurt
  (case 0.2 (strength (u 'distortion)) :from 0.0 :to 0.5 :ease #'easing-f:out-exp)
  (case 0.1 (strength (u 'distortion)) :from 0.5 :to 0.0 :ease #'easing-f:out-exp))

(action-list:define-action-list transition
  (setf (direction (u 'fade)) 0.0)
  (case 0.5 (strength (u 'fade)) :from 0.0 :to 1.0)
  (delay 0.1)
  (eval (loop for func = (pop (on-complete (u 'fade)))
              while func
              do (funcall func))
        (setf (direction (u 'fade)) 1.0))
  (delay 0.1)
  (case 0.5 (strength (u 'fade)) :from 1.0 :to 0.0))

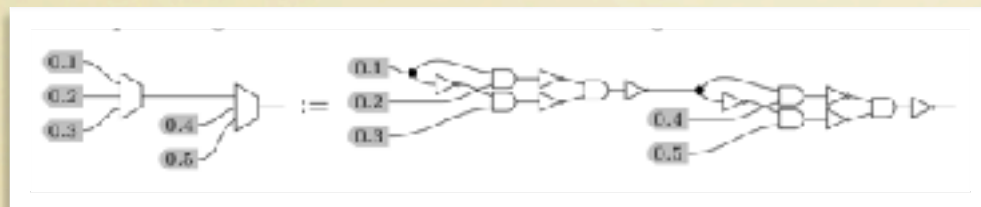
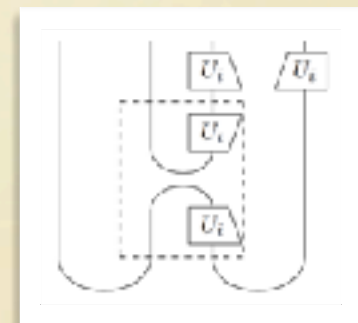
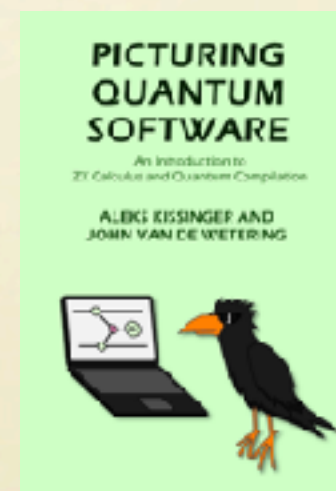
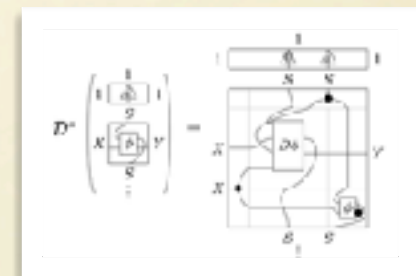
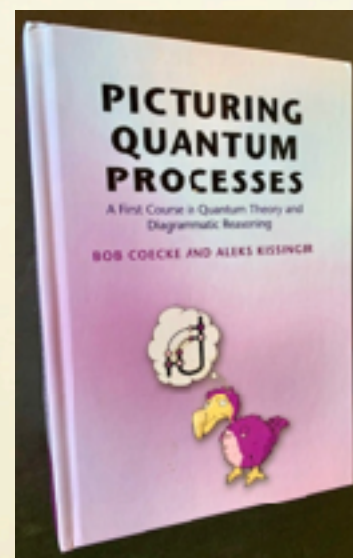
(action-list:define-action-list start-game
```

```
1 # User Defined Function to add Two Numbers
2 """ User Defined Function to add Two Numbers """
3 #
4 def add2nums(a,b):
5     sum = a+b
6     return sum
7 #
8 # Main to call add2nums Function
9 num1 = eval(input('Enter first number: '))
10 num2 = eval(input('Enter second number: '))
11 sum = num1 + num2
12 print(f'Addition sum {num1} + {num2} is = {sum}')
```

[illegible]

The Diagram Renaissance

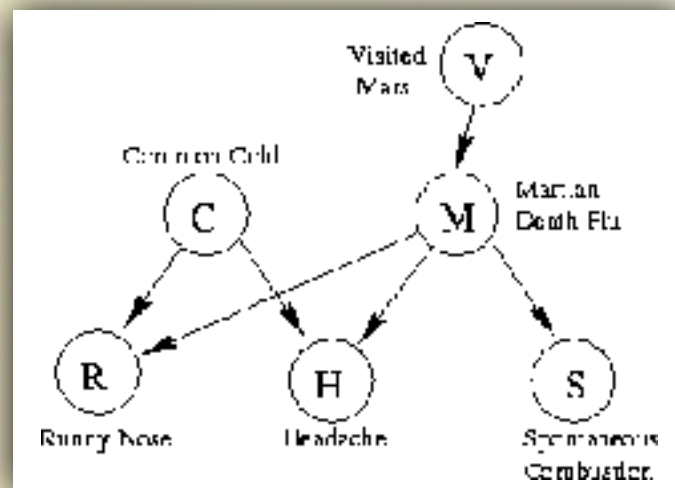
After decades of domination of textual notations for logic reasoning and computing, we are witnessing a **resurgence of diagrammatic formalisms** in Computer Science.



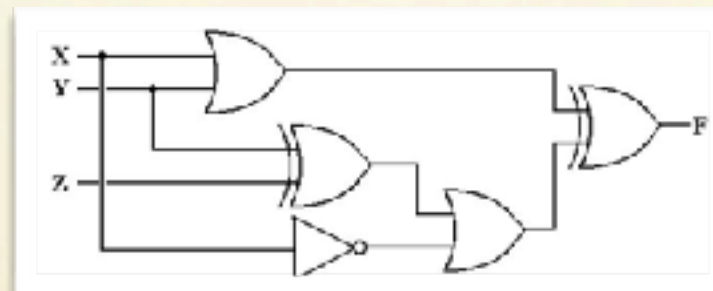
Why?

Resource Sensitive Computation

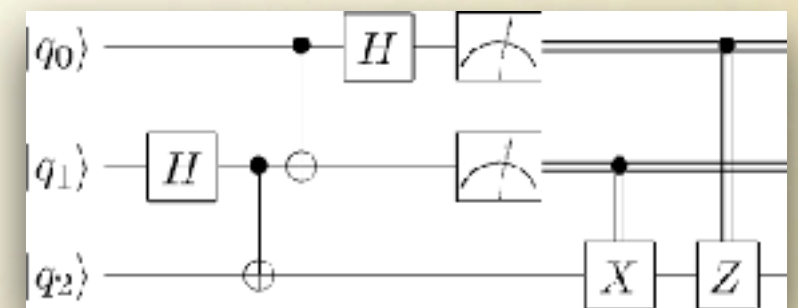
Bayesian networks



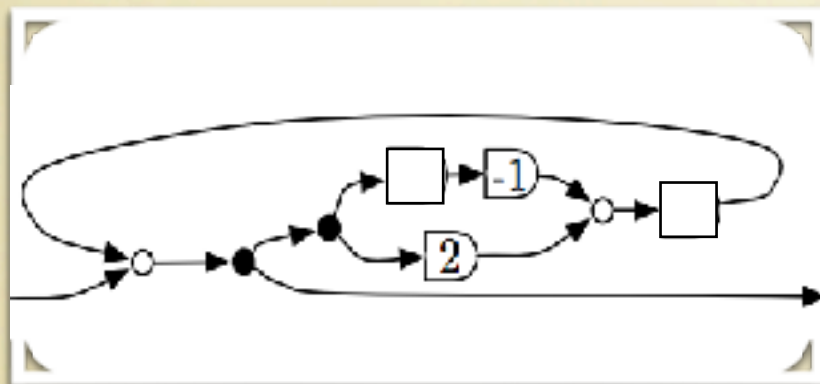
Digital Circuits



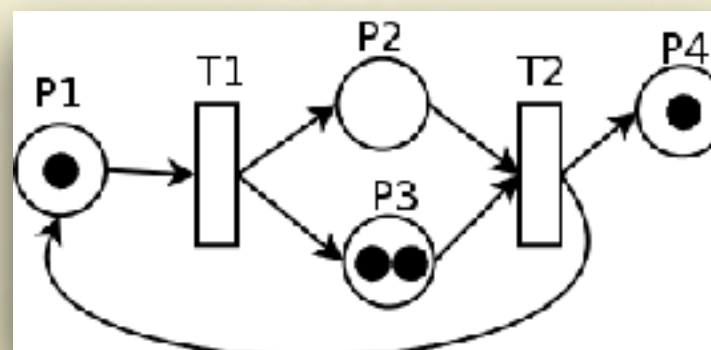
Quantum circuits



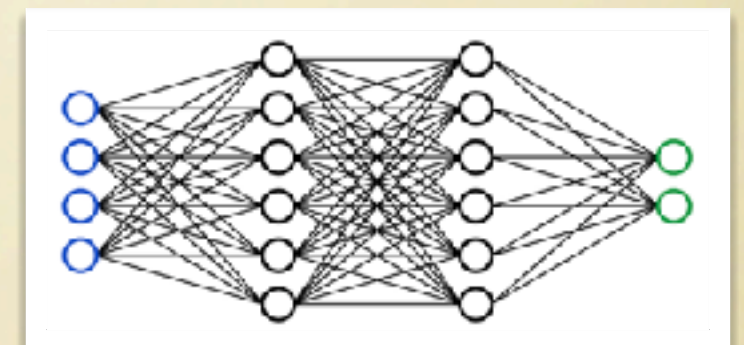
Signal flow graphs



Petri nets



Neural Networks



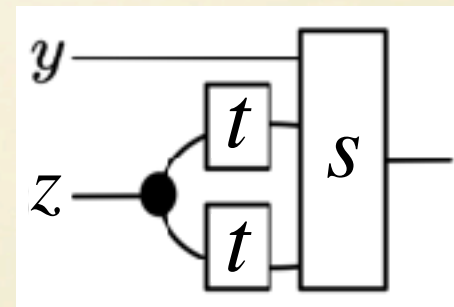
Resource Sensitive Computation

Visual formalisms are particularly **expressive** for representing resource manipulation.

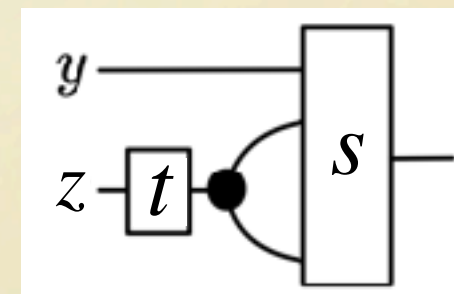
$s(y, t(z), t(z))$

VS

Process t consumes resource z twice

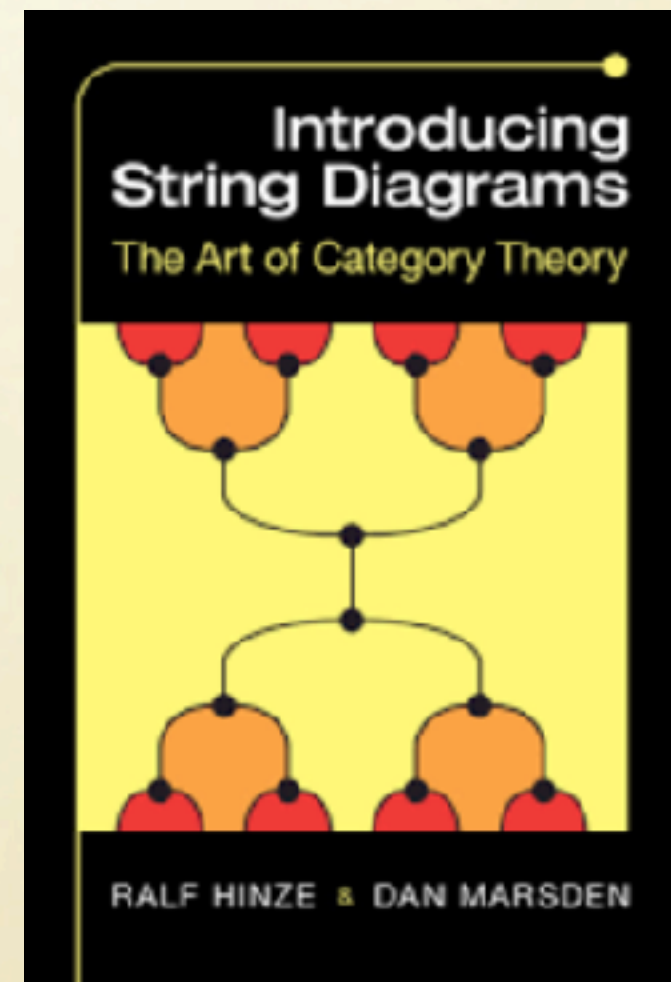
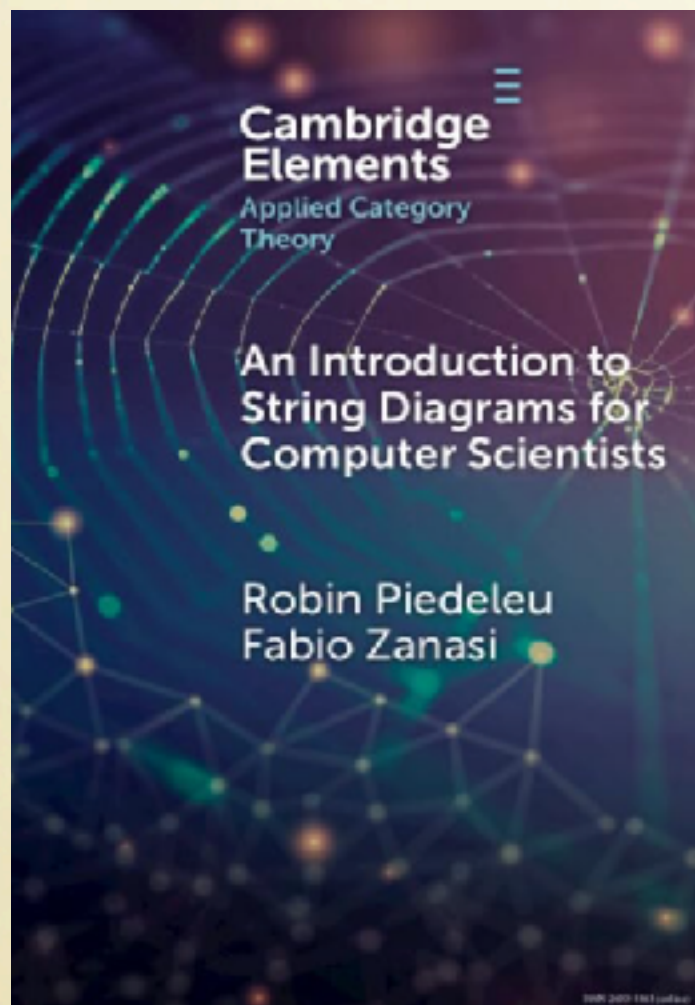


Process t consumes resource z once



New Technology: String Diagrams

A formal language originated in Category Theory, called **String Diagrams**, have provided the mathematical framework underpinning this research.



Peirce Vindicated

Many of Peirce's original insights found a new home in string diagram-based approaches in Theoretical Computer Science.

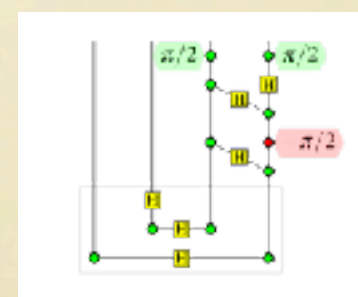
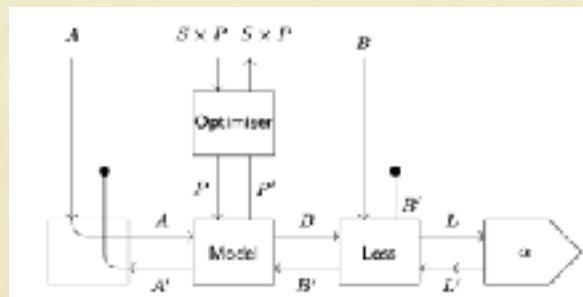
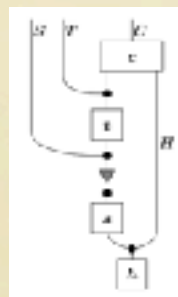
- Relations as unit of reasoning
- Variable-free language
- Computation based on geometric and topological manipulations
- The formal model should reflect the real-world phenomena it refers to.
- The formal model should align with how humans naturally think.

Summing Up

Formal languages in Philosophy, Logic, and Computer Science have privileged textual notation for over a century.

The emergency of resource sensitive paradigms has determined a renewed interest for two-dimensional, visual formalisms.

String diagrams, increasingly widespread in Computer Science, embody a Peircean semiotic philosophy.



Final Thoughts

Final Thoughts

- Computer Science was born out of intense philosophical debate about logic, language, and the nature of reasoning.
- Fundamental CS concepts still carry the imprint of philosophical ideas from Frege, Russell, Lesniewski, Church, Curry, Peirce, and others.
- Being aware of these influences can improve our understanding of formal modelling and how we design programming languages: we need a closer dialogue between philosophers and computer scientists.